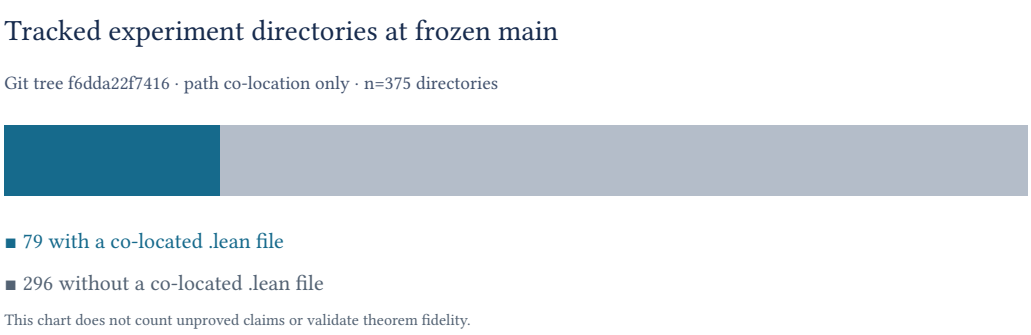
Cryptanalysis formal coverage: frozen path inventory

28 October 2026 · source commit `f6dda22f74168b6578eb47c80a969273db8753ab` · tree `0b84b8882f063b6ac1321250af9a4efb2442409b`

This report inventories tracked paths and lexical Lean declaration pointers. It makes no judgment about whether a Lean statement faithfully represents an experiment result, whether Lean builds, or whether the underlying experiment is valid.



Method and observations

`tools/formal_coverage_inventory.py` runs `git ls-tree -r -z` on the frozen commit and reads every tracked `.lean` blob with `git cat-file --batch`. The local command completed without an API truncation limit. `inventory.json` records exact paths, blob object IDs, and line numbers for lines lexically matching `theorem` or `lemma` declarations.

- 59789 tracked Git blobs; 375 experiment directories.
- 137 `.lean` files across the repository; 125 under `experiments/`.
- 79 experiment directories contain at least one `.lean` file; 296 do not.
- 1027 lexical declaration pointers. Regex extraction can include comments or omit other declaration forms, and does not parse Lean syntax.

Interpretation and next gate

The gray segment is an inspection queue, not a missing-proof count: an operational experiment may have no mathematical proposition to formalize, while a result may be proved in a shared file elsewhere. Review each substantive claim against its exact Lean statement and hypotheses, run the pinned theorem build and axiom audit, then independently validate its empirical evidence before promoting any result. The repository's existing Lean gate also does not establish statement-to-claim fidelity.

Reproduction

From this commit's checkout: `python3 tools/formal_coverage_inventory.py f6dda22f74168b6578eb47c80a969273db8753ab docs/reports/2026-10-28-formal-coverage/inventory.json`, then `python3 docs/reports/2026-10-28-formal-coverage/render.py`. Source data are the frozen Git tree, not a GitHub recursive API response.